

## Übungsblatt 2

### Vorlesung “Cybersicherheit”

Sommersemester 2025

### Praktische Übung Symmetrische Kryptografie

## Vernam-Chipher (Onetimepad und Stromchiffre)

Die Vernam-Chiffre stellt eine Erweiterung der Vigenère-Chiffre dar, bei der für die Verschlüsselung ein Schlüssel mit der gleichen Länge wie der Klartext hat. Ein solches Verfahren gilt nach heutigen Standard als unsicher. Wird aber der Schlüssel durch einen kryptografisch starken Zufallszahlengenerator erzeugt, ist Vernam eine Stromverschlüsselung. Die Sicherheit hängt hier von der Sicherheit des Zufallszahlengenerators ab. Ist der Schlüssel echt zufällig gewählt, so wird es auch One-Time-Pad genannt.

|   |   |   |   |   |    |   |    |   |    |      |
|---|---|---|---|---|----|---|----|---|----|------|
| A | 0 | F | 5 | K | 10 | P | 15 | U | 20 |      |
| B | 1 | G | 6 | L | 11 | Q | 16 | V | 21 |      |
| C | 2 | H | 7 | M | 12 | R | 17 | W | 22 | Z 25 |
| D | 3 | I | 8 | N | 13 | S | 18 | X | 23 |      |
| E | 4 | J | 9 | O | 14 | T | 19 | Y | 24 |      |

Tabelle 1: Caesar Alphabet, assigned to numbers (0-25)

Verschlüsselung:

1. Für die Verschlüsselung wird jedem Buchstaben im Klartext eine Zahl von 0 bis 26 zugeordnet (siehe Tabelle 1).
2. Im zweiten Schritt wird der Buchstabe mit dem Schlüssel addiert. (plaintext + key)
3. Subtrahieren Sie 26, falls die resultierende Zahl größer als 25 ist. Ansonsten bleibt diese unverändert.

Für die Entschlüsselung gilt der umgekehrte Prozess.

Formale Definition:

$Z_{26}$  alle Zahlen von 0 bis 25, das Alphabet wird wie in Tabelle 1 beschrieben eindeutig zugeordnet.  
Sei der Schlüssel  $k \in Z_{26}$   
Verschlüsselung: Für jeden Buchstaben im Klartext wird die folgende Funktion angewandt:  
 $e(x) = (\text{Buchstabe} + k) \bmod 26$   
Entschlüsselung: Für jeden Buchstaben im Klartext wird die folgende Funktion angewandt:  
 $d(y) = (\text{Buchstabe} - k) \bmod 26$

Beispiel:

|    |    |    |    |    |    |   |    |    |    |    |
|----|----|----|----|----|----|---|----|----|----|----|
| C  | Y  | B  | E  | R  | S  | E | C  | U  | R  | E  |
| 2  | 24 | 1  | 4  | 17 | 18 | 4 | 2  | 20 | 17 | 4  |
| K  | E  | Y  | R  | A  | N  | D | O  | M  | K  | I  |
| 10 | 4  | 24 | 17 | 0  | 13 | 3 | 14 | 12 | 10 | 8  |
| M  | C  | Z  | V  | R  | F  | I | Q  | G  | B  | M  |
| 12 | 2  | 25 | 21 | 17 | 5  | 8 | 16 | 6  | 1  | 12 |

### Übung 1.

1. Verschlüsseln Sie die Nachricht: **VORLESUNG** mit der Vernam-Chiffre und dem Key: **SECUNIDUE**?
2. Bei der vorgestellten Methode handelt es sich um eine (schwache) Variante des Onetimepads. Geben Sie die Anzahl der möglichen Schlüssel für eine Vernam-Chiffre mit Klartextlänge **5** an. Wie viele Schlüssel gibt es für einen Klartext der Länge **n**?
3. Angenommen eine CPU mit 2 Kernen und 4 GHz kann vollständig mit der Berechnung von Schlüsseln für einen Text der Länge **1024** ausgelastet werden. Pro Schlüsselberechnung werden 4 Operationen benötigt. 1 Ghz entspricht 1.000.000 Operationen pro Sekunde. Wie lange benötigt ein Angreifer um mit dieser CPU einen passenden Schlüssel zu finden (Brute-Force Angriff)?
4. *Mehrfache Verschlüsselung*: Eine beliebte Methode um die Sicherheit von Verschlüsselungsverfahren zu erhöhen ist das mehrfache Anwenden der Verschlüsselung. So kann die in der Vorlesung vorgestellte **DES** Chiffre durch die dreifache Anwendung sicher gemacht werden.  
Eine doppelte Verschlüsselung kann wie folgt mathematisch dargestellt werden:  $y \equiv e_{k_2}(e_{k_1}(x))$   
Wie verändert die doppelte Anwendung der Vernam-Chiffre den Schlüsselraum  $K$ . Geben Sie eine allgemeine Formel in Abhängigkeit der Nachrichtenlänge **n** an.

### Lösung 1.

|    |                   |    |    |    |    |    |    |    |    |    |
|----|-------------------|----|----|----|----|----|----|----|----|----|
|    | VORLESUNG         | 21 | 14 | 17 | 11 | 4  | 18 | 20 | 13 | 6  |
| 1. | SECUNIDUE         | 18 | 4  | 2  | 20 | 13 | 8  | 3  | 20 | 4  |
|    | Nachricht + Key   | 39 | 18 | 19 | 31 | 17 | 26 | 23 | 33 | 10 |
|    | $\mathbb{Z}_{26}$ | 13 | 18 | 19 | 5  | 17 | 0  | 23 | 7  | 10 |
|    | Chiffre           | N  | S  | T  | F  | R  | A  | X  | H  | K  |

2. Länge 5:  $26 * 26 * 26 * 26 * 26 = 26^5$   
Für den Text der Länge  $n$ :  $26^n$
3.
  - $4 \text{ Ghz} \approx 4 * 10^6$  Operationen
  - Effektiv also  $10^6$  Schlüsselberechnungen per Kern (CPU)
  - $2 \text{ Kerne} = 2 * 10^6 \text{ Operationen}$
  - $26^{1024} \approx 8,56 * 10^{1448}$  Schlüssel
  - $8,56 * 10^{1448} \div (2 * 10^6) \approx 4,28 * 10^{1442} \text{ Sekunden} \approx 1,35 * 10^{1435} \text{ Jahre}$
4. Die doppelte Verschlüsselung hat keinen Nutzen, weil sich die Verschlüsselung mit *Key k3* aus der Summe von  $k1 + k2$  darstellen lässt!  
 $e(x) = (\text{Buchstabe} + k) \bmod 26$   
2-fach verschlüsselt Vernam bedeutet:  $(\text{Buchstabe} + k1) \bmod 26 + (\text{Buchstabe} + k2) \bmod 26$   
Dies ist jedoch gleichwertig zu:  $(\text{Buchstabe} + (k1 + k2)) \bmod 26$ , wobei  $(k1+k2)$ ,  $k3$  ist

## Formbarkeit (engl. Malleability) von Stromchiffren

Die Formbarkeit bezeichnet eine besondere Eigenschaft eines Kryptosystems. Ein Kryptosystem gilt als formbar, wenn Veränderungen am ciphertext (y) vorgenommen werden können, die sich gezielt auf den plaintext (x) auswirken, ohne dass der dazugehörige plaintext bekannt sein muss. Der Angreifer hat die volle Kontrolle über Änderungen am Klartext, obwohl ihm der entsprechende Klartext unbekannt ist.

## Übung 2.

1. Beschreiben Sie, wieso Stromchiffren im Prinzip immer die Eigenschaft eines formbaren Kryptosystems aufweisen und wie ein Angreifer vorgehen muss, um gezielt Änderungen am Klartext vorzunehmen.
2. Sie sind nun ein Angreifer und haben sich über Nacht in das System eines Bitcoin-Brokers gehackt. Der Broker überweist Ihnen am nächsten Morgen 1 Bitcoin (zurzeit ca. 40 000 EUR). Die Anzahl an Bitcoins wird als 16-bit Integer (in C *short*) gespeichert. Wie müssen Sie den Wert manipulieren um über Nacht zum Millionär zu werden?
3. Angenommen Sie sind ein böser Admin bei einem Streaming-Dienst mit kostenlosem Mitarbeiter-Abo. Sie möchten jedoch alle Premium-Angebote ohne Zusatzkosten nutzen. Daher entscheiden Sie sich den Wert von 'm' (Mitarbeiter) zu 'p' (Premium) zu ändern. Die beiden Werte 'm' und 'p' sind ASCII kodiert (siehe Tabelle 1). Der Eintrag in der Datenbank liegt als Chiffre  $y('m')$  (**Stromchiffre**) vor und lautet binär *100 0000*. Geben Sie den Eintrag (Chiffre von 'p') in binärer Darstellung an, den Sie setzen, müssen um mit dem Premium-Paket zu schauen. Bemerkung: Es handelt sich hierbei, um einen sogenannten known-plaintext Angriff.

## Lösung 2.

1. Es findet eine XOR-Addition des Klartexts mit dem Schlüsselstrom statt. Daher kann der Angreifer beliebige Bitstrings auf das Chiffre addieren, die sich direkt auf den Klartext auswirken.
2. Intention: Den Wert, welcher die Anzahl an Bitcoins darstellt direkt anpassen. Der Wert muss auf 25 oder höher gesetzt werden um  $10^6$  Euro in BTC (Bitcoin) zu erhalten. ( $10^6 \div 40\,000 = 25$ ) Dies soll durch eine XOR Operation geschehen:

binär 25: *11001*

binär 1: *00001*

$$\begin{array}{rcccccc}
 & 0 & 0 & 0 & 0 & 1 \\
 \oplus & 1 & 1 & 0 & 0 & 0 \\
 \hline
 & 1 & 1 & 0 & 0 & 1
 \end{array}$$

Es muss *11000* XOR addiert werden um zum Millionär zu werden.

3. Intention:

- dem Admin liegt das Chiffre vor
- der Admin kennt die Position des ASCII Wertes  $m$  im Chiffre
- das  $m$  soll zu  $p$  geändert werden
- es soll ein Wert angegeben werden, welcher dekodiert  $p$  ergibt Sie können einen beliebigen Schlüsselstrom annehmen bzw. ein beliebiges Chiffre, das dem Admin bekannt ist. Hier nehmen wir an das Chiffre für  $m$  ist: *100 000* Zunächst ermitteln wir den Schlüsselstrom  $s$ , da wir Klartext und Chiffre sowie die XOR Operation kennen:

$$\begin{array}{rcccccc}
 m \text{ in ASCII} & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\
 s = & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\
 \hline
 y = & 1 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array}$$

Nun ermitteln wir das Chiffre, welches  $p$  darstellt:

$$\begin{array}{rcccccc}
 p \text{ in ASCII} & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\
 s = & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\
 \hline
 y = & 1 & 0 & 1 & 1 & 1 & 0 & 1
 \end{array}$$

Der Admin muss *1011101* in die Datenbank eintragen um das Premium-Abo nutzen zu können.

Ansatz 2:

Unterschied  $m$  zu  $p$

$$\begin{array}{rcccccc}
 m \text{ in ASCII} & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\
 p \text{ in ASCII} & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\
 \hline
 \text{XOR Differenz} & 0 & 0 & 1 & 1 & 1 & 0 & 1
 \end{array}$$

|               |   |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|---|
| s=            | 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| XOR Differenz | 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| p             | 0 | 1 | 1 | 0 | 0 | 0 | 0 |

Aufgrund der Eigenschaft von XOR und  $\begin{cases} x \oplus s = y \\ y \oplus s = x \end{cases}$

ist es möglich zuerst die Differenz zwischen  $m$  und  $p$  zu berechnen und dann das Chiffre durch XOR Addition mit dem Schlüsselstrom zu bestimmen.

## 1 ASCII Table

| Decimal | Hex | Char                   | Decimal | Hex | Char    | Decimal | Hex | Char | Decimal | Hex | Char  |
|---------|-----|------------------------|---------|-----|---------|---------|-----|------|---------|-----|-------|
| 0       | 0   | [NULL]                 | 32      | 20  | [SPACE] | 64      | 40  | @    | 96      | 60  | `     |
| 1       | 1   | [START OF HEADING]     | 33      | 21  | !       | 65      | 41  | A    | 97      | 61  | a     |
| 2       | 2   | [START OF TEXT]        | 34      | 22  | "       | 66      | 42  | B    | 98      | 62  | b     |
| 3       | 3   | [END OF TEXT]          | 35      | 23  | #       | 67      | 43  | C    | 99      | 63  | c     |
| 4       | 4   | [END OF TRANSMISSION]  | 36      | 24  | \$      | 68      | 44  | D    | 100     | 64  | d     |
| 5       | 5   | [ENQUIRY]              | 37      | 25  | %       | 69      | 45  | E    | 101     | 65  | e     |
| 6       | 6   | [ACKNOWLEDGE]          | 38      | 26  | &       | 70      | 46  | F    | 102     | 66  | f     |
| 7       | 7   | [BELL]                 | 39      | 27  | '       | 71      | 47  | G    | 103     | 67  | g     |
| 8       | 8   | [BACKSPACE]            | 40      | 28  | (       | 72      | 48  | H    | 104     | 68  | h     |
| 9       | 9   | [HORIZONTAL TAB]       | 41      | 29  | )       | 73      | 49  | I    | 105     | 69  | i     |
| 10      | A   | [LINE FEED]            | 42      | 2A  | *       | 74      | 4A  | J    | 106     | 6A  | j     |
| 11      | B   | [VERTICAL TAB]         | 43      | 2B  | +       | 75      | 4B  | K    | 107     | 6B  | k     |
| 12      | C   | [FORM FEED]            | 44      | 2C  | ,       | 76      | 4C  | L    | 108     | 6C  | l     |
| 13      | D   | [CARRIAGE RETURN]      | 45      | 2D  | -       | 77      | 4D  | M    | 109     | 6D  | m     |
| 14      | E   | [SHIFT OUT]            | 46      | 2E  | .       | 78      | 4E  | N    | 110     | 6E  | n     |
| 15      | F   | [SHIFT IN]             | 47      | 2F  | /       | 79      | 4F  | O    | 111     | 6F  | o     |
| 16      | 10  | [DATA LINK ESCAPE]     | 48      | 30  | 0       | 80      | 50  | P    | 112     | 70  | p     |
| 17      | 11  | [DEVICE CONTROL 1]     | 49      | 31  | 1       | 81      | 51  | Q    | 113     | 71  | q     |
| 18      | 12  | [DEVICE CONTROL 2]     | 50      | 32  | 2       | 82      | 52  | R    | 114     | 72  | r     |
| 19      | 13  | [DEVICE CONTROL 3]     | 51      | 33  | 3       | 83      | 53  | S    | 115     | 73  | s     |
| 20      | 14  | [DEVICE CONTROL 4]     | 52      | 34  | 4       | 84      | 54  | T    | 116     | 74  | t     |
| 21      | 15  | [NEGATIVE ACKNOWLEDGE] | 53      | 35  | 5       | 85      | 55  | U    | 117     | 75  | u     |
| 22      | 16  | [SYNCHRONOUS IDLE]     | 54      | 36  | 6       | 86      | 56  | V    | 118     | 76  | v     |
| 23      | 17  | [ENG OF TRANS. BLOCK]  | 55      | 37  | 7       | 87      | 57  | W    | 119     | 77  | w     |
| 24      | 18  | [CANCEL]               | 56      | 38  | 8       | 88      | 58  | X    | 120     | 78  | x     |
| 25      | 19  | [END OF MEDIUM]        | 57      | 39  | 9       | 89      | 59  | Y    | 121     | 79  | y     |
| 26      | 1A  | [SUBSTITUTE]           | 58      | 3A  | :       | 90      | 5A  | Z    | 122     | 7A  | z     |
| 27      | 1B  | [ESCAPE]               | 59      | 3B  | ;       | 91      | 5B  | [    | 123     | 7B  | {     |
| 28      | 1C  | [FILE SEPARATOR]       | 60      | 3C  | <       | 92      | 5C  | \    | 124     | 7C  |       |
| 29      | 1D  | [GROUP SEPARATOR]      | 61      | 3D  | =       | 93      | 5D  | ]    | 125     | 7D  | }     |
| 30      | 1E  | [RECORD SEPARATOR]     | 62      | 3E  | >       | 94      | 5E  | ^    | 126     | 7E  | ~     |
| 31      | 1F  | [UNIT SEPARATOR]       | 63      | 3F  | ?       | 95      | 5F  | _    | 127     | 7F  | [DEL] |

Abbildung 1: ASCII Tabelle

## Die Blockchiffre DES

DES basiert auf der Anwendung von Substitutionsboxen und der Struktur namens Feistelnetzwerk. Bei den S-Boxen für eine sichere Verschlüsselung müssen folgende Anforderungen erfüllt sein, um eine sichere Chiffre zu erhalten:

**Vollständigkeit** Jedes Bit in der Ausgabe ist von jedem Bit in der Eingabe abhängig.

**Avalanche** Die Änderung eines Bits in der Eingabe ändert im Mittel die Hälfte der Ausgabebits

**Nichtlinearität** Kein Ausgangsbit ist linear von einem Eingangsbit abhängig.

**Korrelationsimmunität** Solange nur ein Teil der Eingangsbits bekannt ist können keine Rückschlüsse auf die Ausgangsbits gezogen werden und umgekehrt.

Tabelle 2 zeigt die S-Box #5 von DES. Die S-Boxen von DES wurden so erstellt, dass sie den obigen Kriterien genügen.

## SBox 5 (DES)

| $S_5$       |    | Mittlere 4 Bits des Eingabewertes |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|-------------|----|-----------------------------------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
|             |    | 0000                              | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |
| Äußere Bits | 00 | 0010                              | 1100 | 0100 | 0001 | 0111 | 1010 | 1011 | 0110 | 1000 | 0101 | 0011 | 1111 | 1101 | 0000 | 1110 | 1001 |
|             | 01 | 1110                              | 1011 | 0010 | 1100 | 0100 | 0111 | 1101 | 0001 | 0101 | 0000 | 1111 | 1010 | 0011 | 1001 | 1000 | 0110 |
|             | 10 | 0100                              | 0010 | 0001 | 1011 | 1010 | 1101 | 0111 | 1000 | 1111 | 1001 | 1100 | 0101 | 0110 | 0011 | 0000 | 1110 |
|             | 11 | 1011                              | 1000 | 1100 | 0111 | 0001 | 1110 | 0010 | 1101 | 0110 | 1111 | 0000 | 1001 | 1010 | 0100 | 0101 | 0011 |

Abbildung 2: SBox #5 von DES. Hier wird ein Eingabewert mit 6-Bit angenommen, welcher einen 4-Bit Ausgabewert durch Substitution erzeugt. Beispiel: **011011**, hat die äußeren Bits **01** und die inneren Bits **1101**, folglich ergibt sich laut Tabelle 1001 als Substitution (gelb markiert).

## Fragen zu DES

### Übung 3.

1. Welche Schlüssellänge wurde für DES ursprünglich vorgeschlagen und welche (effektive) Schlüssellänge wurde am Ende gewählt? Wer ist für diese Änderung verantwortlich?
2. Was steckt hinter den Begriffen Konfusion und Diffusion und welche Bausteine setzen diese Funktion beim DES um?
3. Zeichnen Sie ein Schaubild eines Feistelnetzwerks.

### Lösung 3.

1. Ursprünglich *128 bit* vorgesehen, aber auf drängen der NSA auf *56 bit* reduziert.
2.
  - **Konfusion:** Eine Operation, um die Beziehung zwischen Schlüssel und Chiffirat zu verschleiern. Bsp: *Substitutionstabellen*.
  - **Diffusion:** Eine Operation, um den Einfluss eines Klartextsymbols auf zahlreiche Chiffrat-symbole zu streuen. Bsp: *Bitpermutation*.
3. Siehe Vorlesungsfolien

## Nichtlinearität von S-Boxen

| $S_1$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 14 | 04 | 13 | 01 | 02 | 15 | 11 | 08 | 03 | 10 | 06 | 12 | 05 | 09 | 00 | 07 |
| 1     | 00 | 15 | 07 | 04 | 14 | 02 | 13 | 01 | 10 | 06 | 12 | 11 | 09 | 05 | 03 | 08 |
| 2     | 04 | 01 | 14 | 08 | 13 | 06 | 02 | 11 | 15 | 12 | 09 | 07 | 03 | 10 | 05 | 00 |
| 3     | 15 | 12 | 08 | 02 | 04 | 09 | 01 | 07 | 05 | 11 | 03 | 14 | 10 | 00 | 06 | 13 |

Tabelle 2: S-Box # 1 von DES. Die Werte in der Tabelle sind in Dezimaldarstellung!

Verwenden Sie für die folgenden Aufgaben S-Box #1 von DES. Eine Kopie der S-Box finden sie in Tabelle 2

Eine S-Box  $S_i$  verhält sich linear, wenn folgendes gilt:  
 $S_i(x_1) \oplus S_i(x_2) = S_i(x_1 \oplus x_2)$   
wobei der Operator  $\oplus$  ein bitweises XOR darstellt.

#### Übung 4.

1. Eine wichtige Eigenschaft von DES ist die Nichtlinearität von S-Boxen. Zeigen Sie für folgenden Eingabebits, dass die S-Box nicht linear ist.

- $x_1 = 000000$  und  $x_2 = 000001$
- $x_1 = 111111$  und  $x_2 = 100000$
- $x_1 = 101010$  und  $x_2 = 010101$

#### Lösung 4.

- $S_1(000000) \oplus S_1(000001) = 1110 \neq S_1(000000 \oplus 000001) = 0000$
- $S_1(111111) \oplus S_1(100000) = 1001 \neq S_1(111111 \oplus 100000) = 1000$
- $S_1(101010) \oplus S_1(010101) = 1010 \neq S_1(101010 \oplus 010101) = 1101$

#### Bonusaufgabe: Schwache Keys

Diese Aufgabe benötigt Zeit und einen guten Kenntnis von DES. Daher werden wir diese Aufgabe erst am Anfang der nächsten Übung besprechen.

Ein DES-Key (DES Schlüssel) wird als schwach bezeichnet, wenn die Ver- und Entschlüsselung identische Operationen sind:

Ein DES-Key ist schwach wenn folgende Bedingung erfüllt ist:  
 $DES_{K_w}(x) = (DES_{K_w}(x))^{-1}$ , für alle  $x$ .

**Tipp:** Schauen Sie sich die Subkeys bzw. auch die Rundenschlüssel von DES genau an. Zeichnen Sie den Aufbau von DES für mindestens 2 Runden (Siehe Aufgabe 3).

#### Übung 5.

1. Beschreiben Sie welche Werte die Subkeys  $K_i$  für die 16 DES Runden annehmen müssten, damit die obige Gleichung für schwache Keys erfüllt ist.
2. Es gibt 4 schwache DES-Keys, welche sind dies?
3. Was ist die Wahrscheinlichkeit, dass ein zufällig gewählter DES-Key, ein schwacher DES-Key ist?